TRƯỜNG ĐẠI HỌC CẦN THƠ

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO

PHÁT TRIỂN ỨNG DỤNG WEB

CT449

Giảng viên hướng dẫn:
ThS Nguyễn Minh Trung

Sinh viên thực hiện:
Tên:
Lư Thanh Tùng
MSSV: B1906607

Cần Thơ, năm 2026

Mục Lục

BACKEND 1 ..... **1**

I. Cài đặt môi trường node và git **1**

II. Tạo thư mục Node **1**

III. Quản lý dự án bởi git và Github **2**

IV. Cấu hình Visual Studio Code, Eslint, Prettier **4**

V. Cài đặt Express **5**

VI. Định nghĩa controlers và các route **8**

VII. Cài đặt xử lý lỗi **12**

ĐƯỜNG LINK REPO LAB1: **14**

BACKEND 2 ..... **15**

I. Cài đặt MongoDB **15**

II. Cài đặt thư viện mongodb, định nghĩa hàm trợ giúp kết nối và lớp dịch vụ truy xuất cơ sở dữ liệu (CSDL) **15**

III. Cài đặt các handler **18**

ĐƯỜNG LINK REPO LAB2: **29**

Danh mục hình

Hình 1: Kiểm tra môi trường 1
Hình 2: Tải package cần thiết 1
Hình 3: Kiểm tra bằng lệnh git status 2
Hình 4: Cho git quản lý 2
Hình 5: Các bước push dự án lên github 3
Hình 6: Dự án đã được lưu trên github 3
Hình 7: Tải extention ESLint 4
Hình 8: Tải extentions Prettier – Code formatter 4
Hình 9: Cấu hình VS Code 4
Hình 10: Cài gói eslint và prettier 5
Hình 11: Viết nội dung tập tin .eslint.js 5
Hình 12: Viết nội dung tập tin app.js 5
Hình 13:Viết nội dung tập tin server.js 6
Hình 14: Nội dung file index.js trong thư mục config 6
Hình 15: Thực hiện lệnh chạy server 6
Hình 16: Kết quả thực hiện 6
Hình 17: Cài thêm modemon 7
Hình 18: Thay đổi cấu trức thư mục script trong file package.json 7
Hình 19: Lưu lên git 7
Hình 20: Tạo tập tập tin contact.controller.js 8
Hình 21: Tạo tập tin contact.route.js 8
Hình 22: Hiệu chỉnh tập tin app.js 9
Hình 23:Chạy kết quả thực hiện Định nghĩa controlers và các route 9
Hình 24: Kết quả ở postman GET http://localhost:3000/api/contacts/favorite 9
Hình 25: Kết quả ở postman GET http://localhost:3000/api/contacts 10
Hình 26: Kết quả ở postman POST http://localhost:3000/api/contacts 10
Hình 27: Kết quả ở postman DELETE http://localhost:3000/api/contacts 10
Hình 28: Kết quả ở postman GET http://localhost:3000/api/contacts/1 10
Hình 29: Kết quả ở postman PUT http://localhost:3000/api/contacts/1 10
Hình 30: Kết quả ở postman DELETE http://localhost:3000/api/contacts/1 11
Hình 31: Lưu lên git 11
Hình 32: Tạo tập tin app/api-error.js 12
Hình 33: Thêm middleware vào file app.js 12
Hình 34: Kết quả cài đặt xử lý lỗi trên web 13
Hình 35: Kết quả cài đặt xử lý lỗi trên postman 13
Hình 36: Lưu lên git 13
Hình 37: Cấu trúc thư mục 14
Hình 38: Cài đặt MongoDB 15
Hình 39: install mongodb 15

Hình 40: Hiệu chỉnh tập tinh index.js 16
Hình 41: Tạo tập tin app/utils/mongodb.util.js 16
Hình 42: Thay đổi nội dung server.js 17
Hình 43: Định nghĩa lớp dịch vụ ContactService 17
Hình 44: Hiệu chỉnh tập tin app/controllers/contact.controller.js 18
Hình 45: Định nghĩa phương thức create() 18
Hình 46: Hiêu chinh Header "Content-Type: application/json" 19
Hình 47: Đặt dữ liệu JSON trong phần Body 19
Hình 48: Cài đặt handler findAll 19
Hình 49: Test thử findAll 20
Hình 50: contactService.find(condition) và contactService.findByName(name) 20
Hình 51: Test tìm theo tên 21
Hình 52: Cài đặt handler findOne 21
Hình 53: Phương thức findByld(id) 21
Hình 54: Test tìm theo id 22
Hình 55: Cài đặt handler update 22
Hình 56: Phương thức update(id, document) 23
Hình 57: Test update 23
Hình 58: Cài đặt handler delete 24
Hình 59: Phương thức delete(id) 25
Hình 60: Test Delete 25
Hình 61: Cài đặt handler findAllFavorite 26
Hình 62: Phương thức findFavorite() 26
Hình 63: Test findFavorite 26
Hình 64: Cài đặt handler deleteAll 27
Hình 65: Phương thức deleteAll() 27
Hình 66: Test deleteAll 27
Hình 67: Lưu lên git 28
Hình 68: Đã lưu lên git 28
Hình 69: Cấu trúc thư mục dự án 29

BACKEND 1

I. Cài đặt môi trường node và git

- Thực hiện tải nodejs và git.
- Sau đó kiểm tra bằng lệnh như ảnh:

```
v22.19.0

C:\Users\ACER>git --version
git version 2.51.0.windows.1
```

Hình 1: Kiểm tra môi trường

II. Tạo thư mục Node

- Tạo thư mục dự án: backend1.
- Trong thư mục này, ta khởi tạo ứng dụng Node

```
About to write to C:\Users\ACER\backend1\package.json:

{
  "name": "backend1",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Gia Bao",
  "license": "ISC"
}


Is this OK? (yes) yé


C:\Users\ACER\backend1>npm install express cors

added 70 packages, and audited 71 packages in 2s

16 packages are looking for funding
  run `npm fund` for details
```

Hình 2: Tải package cần thiết

III. Quản lý dự án bởi git và Github:

- Tạo tập tin .gitignore: npx gitignore node . Tập tin *.gitignore* liệt kê các thư mục, tập tin sẽ không được quản lý bởi git (ví dụ như thư mục *node_modules)*.
- Trong thư mục gốc dự án, chạy lệnh git init. Gõ tiếp git status, chúng ta sẽ thấy có 3 tập tin chưa được quản lý bởi git là *.gitignore*, *package*-lock.json và package.json.

```
C:\Users\ACER\backend1>npx gitignore node
Created .gitignore file for flag type node.

C:\Users\ACER\backend1>git init
Initialized empty Git repository in C:/Users/ACER/backend1/.git/

C:\Users\ACER\backend1>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        package-lock.json
        package.json

nothing added to commit but untracked files present (use "git add" to track)
```

Hình 3: Kiểm tra bằng lệnh git status

- Cho git quản lý 3 tập tin .gitignore, package-lock.json và package.json

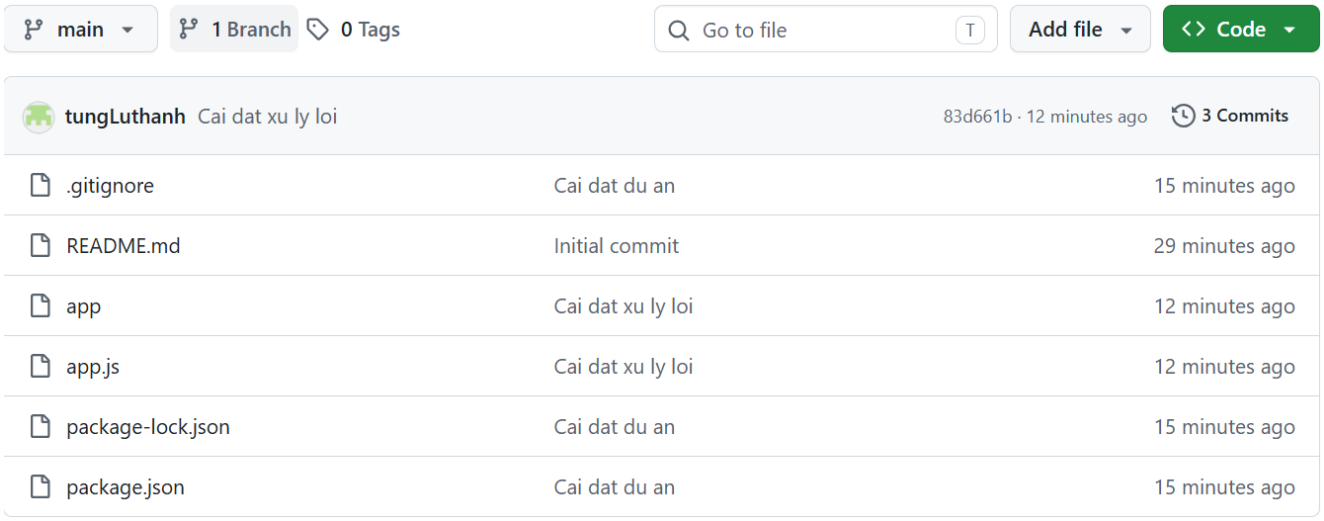
```
C:\Users\ACER\backend1>git add .gitignore package.json package-lock.json
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it

C:\Users\ACER\backend1>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   package-lock.json
        new file:   package.json
```

Hình 4: Cho git quản lý

- push dự án lên github

```
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 2 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (9/9), 21.85 KiB | 3.64 MiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/tungLuthanh/B1906607_LuThanhTung_Backend_1
```

Hình 5: Các bước push dự án lên github

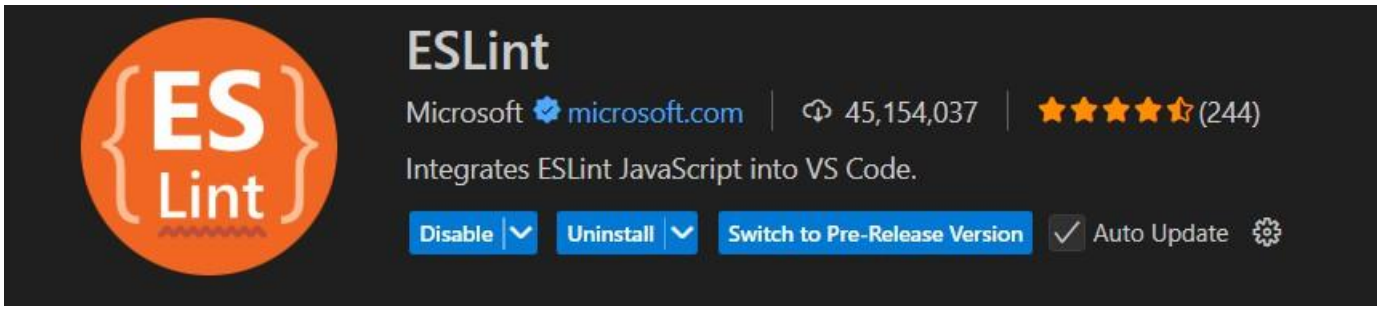


- Dự án đã được lưu trên github

Hình 6: Dự án đã được lưu trên github

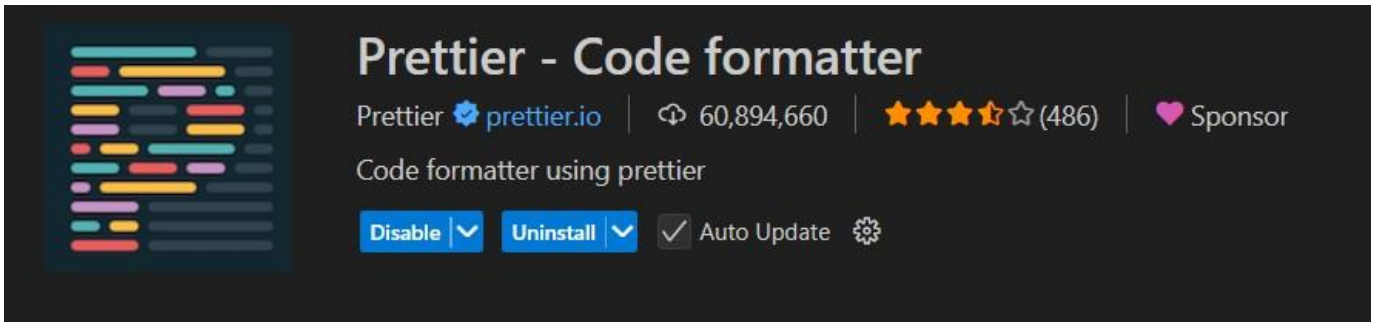
IV. Cấu hình Visual Studio Code, Eslint, Prettier

➢ Cài đặt **ESLint extension** cho VSCode.



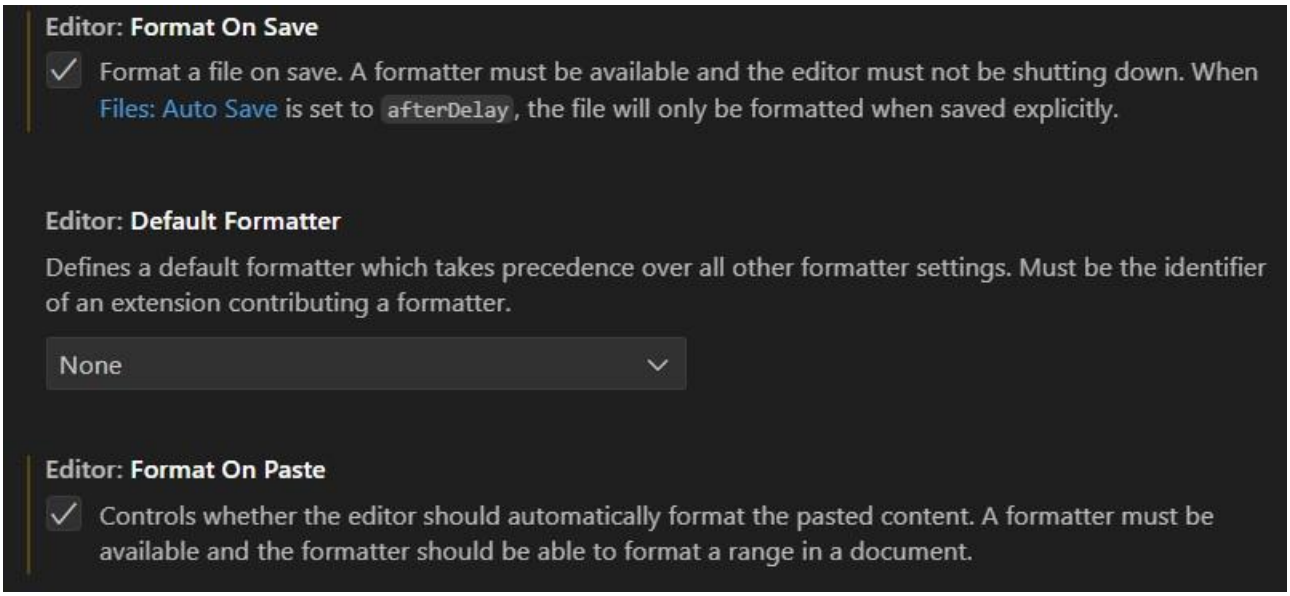
Hình 7: Tải extention ESLint

➢ Tải extentions Prettier – Code formatter



Hình 8: Tải extentions Prettier – Code formatter

➢ Cấu hình VS CODE



Hình 9: Cấu hình VS Code

➢ Cài gói eslint và prettier

```
C:\Users\ACER\backend1>npm i -D eslint prettier eslint-config-prettier

added 85 packages, and audited 156 packages in 4s

40 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

➢

Hình 10: Cài gói eslint và prettier

Viết nội dung tập tin .eslint.js

```
.eslintrc.js U ×
.eslintrc.js > ...
1  module.exports = {
2    env: {
3      commonjs: true,
4      es2021: true,
5      node: true,
6    },
7    extends: ['eslint:recommended', 'prettier'],
8
9  };
```

Hình 11: Viết nội dung tập tin .eslint.js

V. Cài đặt Express

➢ Trong thư mục gốc của dự án, tạo tập tin *app.js* và *server.js* lần lượt với nội dung sau:

- app.js

```
JS app.js > ...
1  const express = require("express");
2  const cors = require("cors");
3
4  const app = express();
5
6  app.use(cors());
7  app.use(express.json());
8
9
10 app.get("/", (req, res) => {
11   res.json({ message: "Welcome to contact book application." });
12 });
13
14 module.exports = app;
```

Hình 12: Viết nội dung tập tin app.js

•

Server.js

```
server.js > ...
const app = require("./app");
const config = require("./app/config");

// start server
const PORT = config.app.port;
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}.`);
});
```

Hình 13:Viết nội dung tập tin server.js

➢

Lần lượt tạo thư mục *app,* app/config và tập tin *app*/config/index.js.

```
app > config > index.js > ...
const config = {
  app: {
    port: process.env.PORT || 3000,
  },
};

module.exports = config;
```

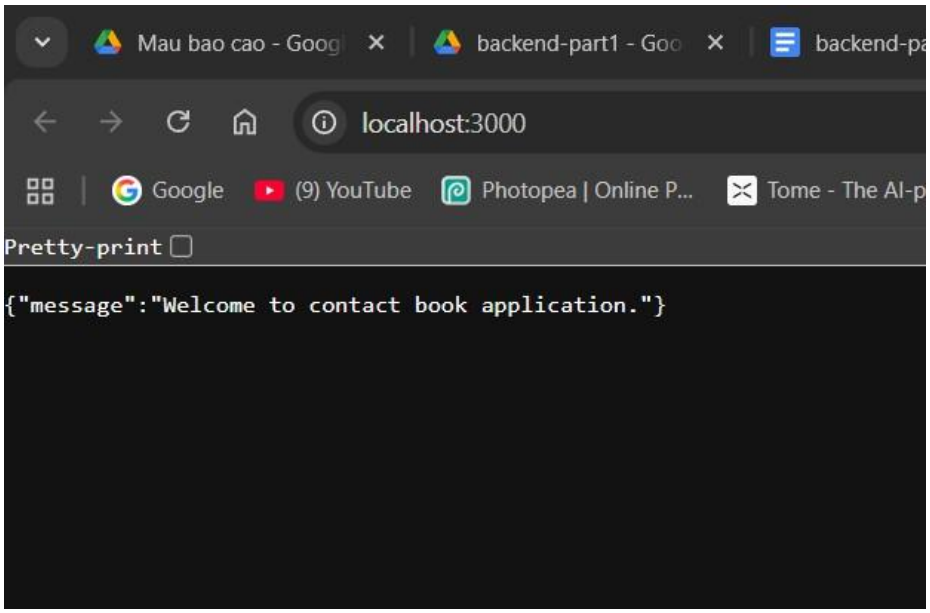
Hình 14: Nội dung file index.js trong thư mục config

➢ Thực hiện lệnh chạy server

```
C:\Users\ACER\backend1>node server.js
Server is running on port 3000.
```

Hình 15: Thực hiện lệnh chạy server

➢ Kết quả thực hiện



Hình 16: Kết quả thực hiện

➢ Cài thêm modemon

```
C:\Users\ACER\backend1>npm install --save-dev nodemon

added 21 packages, and audited 177 packages in 2s

44 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Hình 17: Cài thêm modemon

➢ Mở tập tin package.json và thay đổi cấu hình mục **"scripts"** như sau:

```
"scripts": {
"start": "nodemon server.js",
 "test": "echo \"Error: no test specified\" && exit 1"
},
```

Hình 18: Thay đổi cấu trức thư mục script trong file package.json

➢ Lưu lên git

```
C:\Users\ACER\backend1>git add -u
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it

C:\Users\ACER\backend1>git add app/ app.js server.js

C:\Users\ACER\backend1>git commit -m "Cai dat express chay thong bao xin chao"
[master c399efe] Cai dat express chay thong bao xin chao
 5 files changed, 318 insertions(+)
 create mode 100644 app.js
 create mode 100644 app/config/index.js
 create mode 100644 server.js

C:\Users\ACER\backend1>git push origin master
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), 4.00 KiB | 1.33 MiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/HoLyGB/backend1.git
   14602de..c399efe  master -> master
```

Hình 19: Lưu lên git

VI. Định nghĩa controlers và các route

- Tạo thư mục *controllers* trong thư mục *app*. Sau đó tạo tập tin *contact.controllerjs* trong thư mục controllers với nội dung sau:

```
app > controllers > JS contact.controller.js > ...
exports.create = (req, res) => {
  res.send({ message: "create handler" });
};
exports.findAll = (req, res) => {
  res.send({ message: "findAll handler" });
};

exports.findOne = (req, res) => {
  res.send({ message: "findOne handler" });
};

exports.update = (req, res) => {
  res.send({ message: "update handler" });
};

exports.delete = (req, res) => {
  res.send({ message: "delete handler" });
};

exports.deleteAll = (req, res) => {
  res.send({ message: "deleteAll handler" });
};

exports.findAllFavorite = (req, res) => {
  res.send({ message: "findAllFavorite handler" });
};
```

Hình 20: Tạo tập tập tin contact.controller.js

- Tạo thư mục *routes* trong thư mục *app*. Sau đó tạo tập tin *contact.route.js* trong thư mục *routes* với nội dung sau:

```
app > routes > JS contact.route.js > ...
const express = require("express");
const contacts = require("../controllers/contact.controller");

const router = express.Router();

router.route("/")
  .get(contacts.findAll)
  .post(contacts.create)
  .delete(contacts.deleteAll);

router.route("/favorite")
  .get(contacts.findAllFavorite);

router.route("/:id")
  .get(contacts.findOne)
  .put(contacts.update)
  .delete(contacts.delete);

module.exports = router;
```

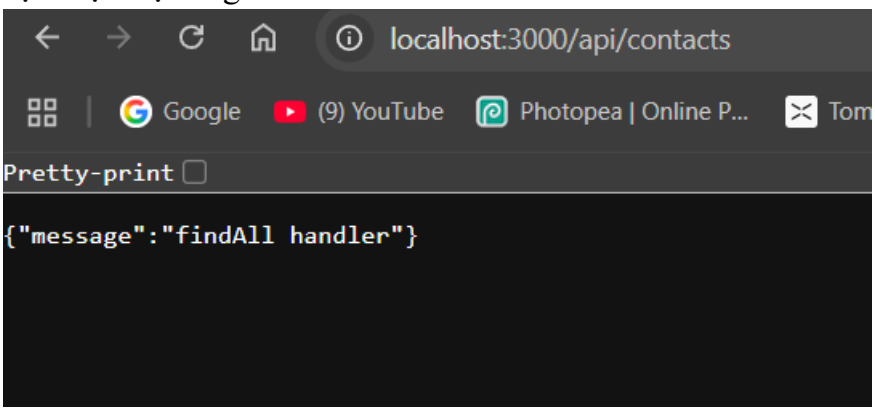
Hình 21: Tạo tập tin contact.route.js

➢ Tiếp theo, chúng ta sẽ đăng ký các route vừa tạo với express app. Hiệu chỉnh tập tin *app.js* như sau:

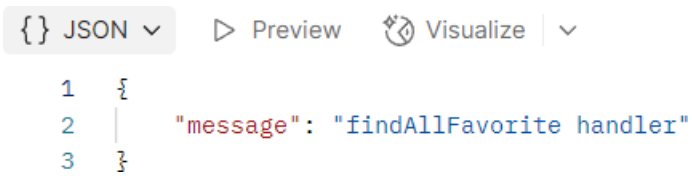
```js
app.js > ...
const express = require("express");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());


const contactsRouter = require("./app/routes/contact.route");
app.use("/api/contacts", contactsRouter);

app.get("/", (req, res) => {
  res.json({ message: "Welcome to contact book application." });
});

module.exports = app;
```

Hình 22: Hiệu chỉnh tập tin app.js

➢ Chạy kết quả thực hiện Định nghĩa controlers và các route



Hình 23:Chạy kết quả thực hiện Định nghĩa controlers và các route



Hình 24: Kết quả ở postman GET http://localhost:3000/api/contacts/favorite

Body Cookies Headers (8) Test Results

{} JSON Preview Visualize

```
1 {
2     "message": "findAll handler"
3 }
```

Hình 25: Kết quả ở postman GET http://localhost:3000/api/contacts

Body Cookies Headers (8) Test Results

{} JSON Preview Visualize

```
1 {
2     "message": "create handler"
3 }
```

Hình 26: Kết quả ở postman POST http://localhost:3000/api/contacts

Body Cookies Headers (8) Test Results

{} JSON Preview Visualize

```
1 {
2     "message": "deleteAll handler"
3 }
```

Hình 27: Kết quả ở postman DELETE http://localhost:3000/api/contacts

Body Cookies Headers (8) Test Results

{} JSON Preview Visualize

```
1 {
2     "message": "findOne handler"
3 }
```

Hình 28: Kết quả ở postman GET http://localhost:3000/api/contacts/1

Body Cookies Headers (8) Test Results

{} JSON Preview Visualize

```
1 {
2     "message": "update handler"
3 }
```

Hình 29: Kết quả ở postman PUT http://localhost:3000/api/contacts/1

Body Cookies Headers (8) Test Results

{} JSON Preview Visualize

```
1 {
2     "message": "delete handler"
3 }
```

Hình 30: Kết quả ở postman DELETE http://localhost:3000/api/contacts/1

➢ Lưu lên git

```
C:\Users\ACER\backend1>git add -u
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it

C:\Users\ACER\backend1>git add app/controllers app/routes

C:\Users\ACER\backend1>git commit -m "Dinh nghia cac route cho tai nguyen Contact"
[master 93ab65e] Dinh nghia cac route cho tai nguyen Contact
 4 files changed, 50 insertions(+), 1 deletion(-)
 create mode 100644 app/controllers/contact.controller.js
 create mode 100644 app/routes/contact.route.js

C:\Users\ACER\backend1>git push origin master
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), 1.06 KiB | 542.00 KiB/s, done.
Total 9 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/HoLyGB/backend1.git
   c399efe..93ab65e  master -> master
```

Hình 31: Lưu lên git

VII. Cài đặt xử lý lỗi

➢ Tạo tập tin *app/api*-error.js.

```
app > JS api-error.js > ...
class ApiError extends Error {
  constructor(statusCode, message) {
    super();
    this.statusCode = statusCode;
    this.message = message;
  }
}

module.exports = ApiError;
```

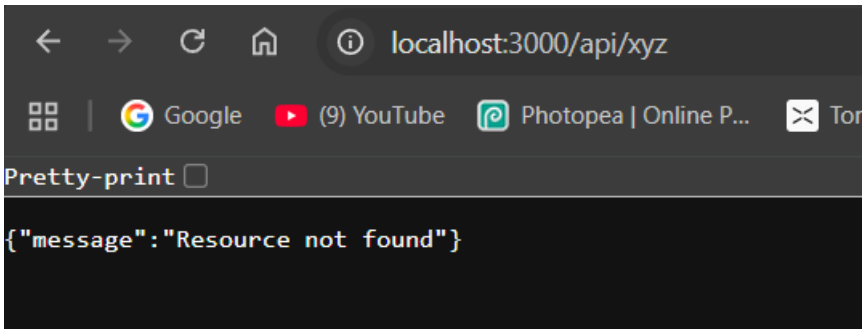
Hình 32: Tạo tập tin app/api-error.js

➢ Mở tập tin *app.js* và thêm vào các middleware xử lý lỗi

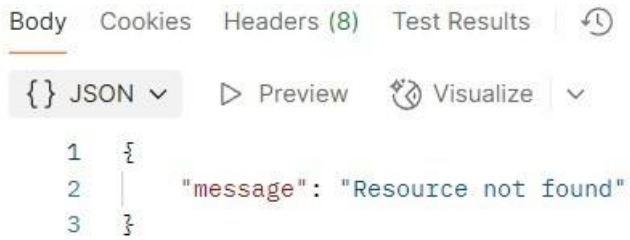
```
JS app.js > app.use() callback > message
const express = require("express");
const cors = require("cors");
const ApiError = require("./app/api-error");

const app = express();

app.use(cors());
app.use(express.json());


const contactsRouter = require("./app/routes/contact.route");
app.use("/api/contacts", contactsRouter);


app.use((req, res, next) => {

  return next(new ApiError(404, "Resource not found"));
});


app.use((err, req, res, next) => {
  return res.status(err.statusCode || 500).json({
    message: err.message || "Internal Server Error",
  });
});

module.exports = app;
```

Hình 33: Thêm middleware vào file app.js

➢ Kiểm tra nhận được lỗi 404 và thông báo "Resource not found"



Hình 34: Kết quả cài đặt xử lý lỗi trên web



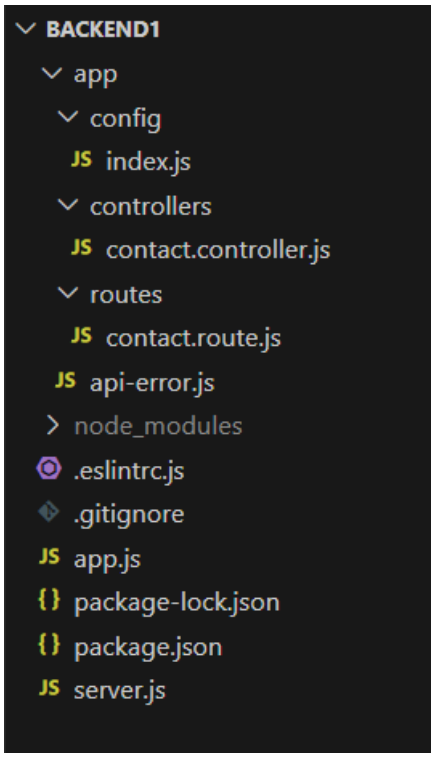
Hình 35: Kết quả cài đặt xử lý lỗi trên postman

➢ Lưu lên git

```
C:\Users\ACER\backend1>git add -u

C:\Users\ACER\backend1>git add app/api-error.js

C:\Users\ACER\backend1>git commit -m "Cai dat xu ly loi"
[master 4b8a307] Cai dat xu ly loi
 2 files changed, 21 insertions(+), 2 deletions(-)
 create mode 100644 app/api-error.js

C:\Users\ACER\backend1>git push origin master
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 837 bytes | 837.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/HoLyGB/backend1.git
   93ab65e..4b8a307  master -> master
```

Hình 36: Lưu lên git

➢ Cấu trúc thư mục

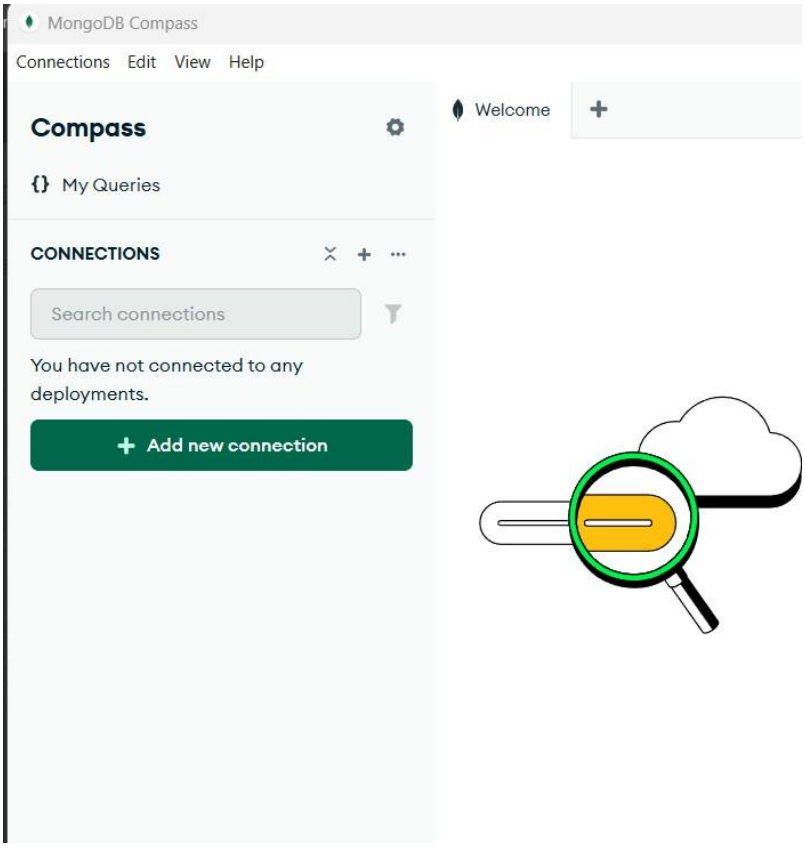


Hình 37: Cấu trúc thư mục

ĐƯỜNG LINK REPO LAB1:

BACKEND 2

I. Cài đặt MongoDB



Hình 38: Cài đặt MongoDB

II. Cài đặt thư viện mongodb, định nghĩa hàm trợ giúp kết nối và lớp dịch vụ truy xuất cơ sở dữ liệu (CSDL)

- Cài đặt thư viện mongodb vào dự án: npm install mongodb

```
C:\Users\ACER\backend1>npm install mongodb

added 11 packages, and audited 188 packages in 3s

44 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Hình 39: install mongodb

➢ Trong thư mục *app*/config, hiệu chỉnh (thêm đoạn code) tập tin *index.js*.

```
app > config > JS index.js > ...
1
2   const config = {
3       app: {
4           port: process.env.PORT || 3000,
5       },
6       db: {
7           uri: process.env.MONGODB_URI || "mongodb://127.0.0.1:27017/contactbook",
8       },
9   };
10
11  module.exports = config;
12
```

Hình 40: Hiệu chỉnh tập tinh index.js

➢ Định nghĩa lớp trợ giúp kết nối đến MongoDB: *app/utils/mongodb.util.js*

```
app > utils > JS mongodb.util.js > ...
1   const { MongoClient } = require("mongodb");
2
3   class MongoDB {
4       static client;
5
6       static connect = async (uri) => {
7           if (this.client) return this.client;
8           this.client = await MongoClient.connect(uri, {
9               useNewUrlParser: true,
10              useUnifiedTopology: true,
11          });
12          return this.client;
13      };
14  }
15
16  module.exports = MongoDB;
17
```

Hình 41: Tạo tập tin app/utils/mongodb.util.js

➢ Thực hiện kết nối đến CSDL MongoDB khi chạy server, thay toàn bộ nội dung tập tin *server.js* bằng đoạn mã sau đây

```
JS server.js > ...
1  const app = require("./app");
2  const config = require("./app/config");
3  const MongoDB = require("./app/utils/mongodb.util");
4
5  async function startServer() {
6      try {
7          await MongoDB.connect(config.db.uri);
8          console.log("✅ Connected to the database!");
9
10         const PORT = config.app.port;
11         app.listen(PORT, () => {
12             console.log(`🚀 Server is running on port ${PORT}`);
13         });
14     } catch (error) {
15         console.log("❌ Cannot connect to the database!", error);
16         process.exit();
17     }
18 }
19
20 startServer();
```

Hình 42: Thay đổi nội dung server.js

➢ Định nghĩa lớp dịch vụ *ContactService* (trong tập tin *app*/services/contact.service.js) chứa các API của thư viện mongodb để thực hiện các thao tác với CSDL MongoDB

```
app > services > JS contact.service.js > ContactService
1  const { ObjectId } = require("mongodb");
2  class ContactService {
3      constructor(client) {
4          this.Contact = client.db().collection("contacts");
5      }
6      // Tạo một document mới
7      async create(payload) {···
10     }
11     // Tìm theo ID
12     async findById(id) {···
16     }
17     // Lấy tất cả
18     async findAll() {···
20     }
21     // Cập nhật theo ID
22     async update(id, payload) {···
29     }
30     // Xóa theo ID
31     async delete(id) {···
36     }
37 }
38 module.exports = ContactService;
39
```

Hình 43: Định nghĩa lớp dịch vụ ContactService

III. Cài đặt các handler

Cài đặt handler create

- Hiệu chỉnh tập tin *app/*controllers/*contact.controller.js*

```
// Create and Save a new Contact
exports.create = async (req, res, next) => {
    if (!req.body?.name) {
        return next(new ApiError(400, "Name can not be empty"));
    }

    try {
        const contactService = new ContactService(MongoDB.client);
        const document = await contactService.create(req.body);
        return res.send(document);
    } catch (error) {
        return next(
            new ApiError(500, "An error occurred while creating the contact")
        );
    }
};
```

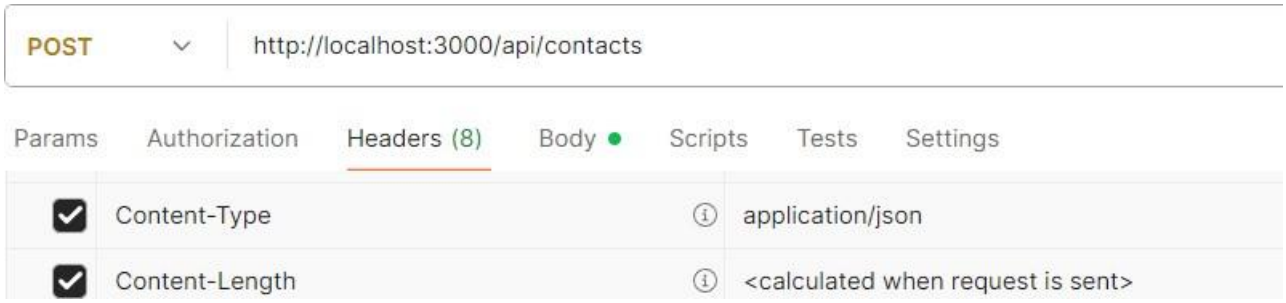
Hình 44: Hiệu chỉnh tập tin app/controllers/contact.controller.js

- Lời gọi contactService.create*()* lưu thông tin đối tượng contact xuống CSDL. Phương thức *create()* được định nghĩa trong lớp ContactService *(*app/services/contact.service.js*)* như sau:

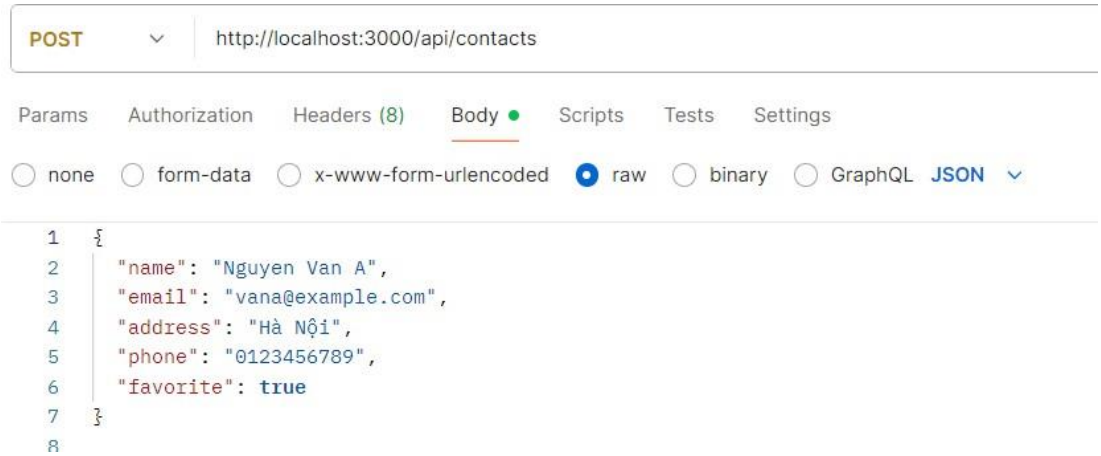
```
app > services > contact.service.js > ContactService > extractContactData > contact
1   const { ObjectId } = require("mongodb");
2
3   class ContactService {
4       constructor(client) {
5           this.Contact = client.db().collection("contacts");
6       }
7
8       extractContactData(payload) {
9           const contact = {
10              name: payload.name,
11              email: payload.email,
12              address: payload.address,
13              phone: payload.phone,
14              favorite: payload.favorite,
15          };
16
17          // Xóa các field undefined
18          Object.keys(contact).forEach(
19              (key) => contact[key] === undefined && delete contact[key]
20          );
21          return contact;
22      }
23
24      async create(payload) {
25          const contact = this.extractContactData(payload);
26          const result = await this.Contact.findOneAndUpdate(
27              contact,
28              { $set: { favorite: contact.favorite === true } },
29              { returnDocument: "after", upsert: true }
30          );
31          return result.value;
32      }
33
```

Hình 45: Định nghĩa phương thức create()

➢ Để gửi dữ liệu JSON về server với Postman hoặc curl, cần đặt Header "Content-Type: application/json" và đặt dữ liệu JSON trong phần Body của yêu cầu



Hình 46: Hiêu chinh Header "Content-Type: application/json"

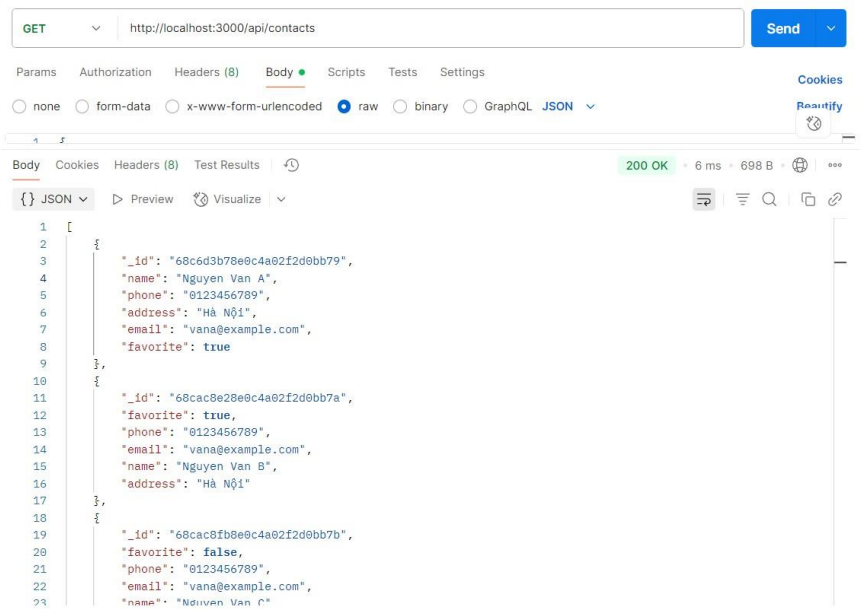


Hình 47: Đặt dữ liệu JSON trong phần Body

➢ Cài đặt handler findAll trong tập tin app/controllers/contact.controllerjs như sau:

```
// Retrieve all contacts of a user from the database
exports.findAll = async (req, res, next) => {
    let documents = [];

    try {
        const contactService = new ContactService(MongoDB.client);
        const { name } = req.query;
        if (name) {
            documents = await contactService.findByName(name);
        } else {
            documents = await contactService.find({});
        }
        return res.send(documents);
    } catch (error) {
        return next(
            new ApiError(500, "An error occurred while retrieving contacts")
        );
    }
};
```

Hình 48: Cài đặt handler findAll

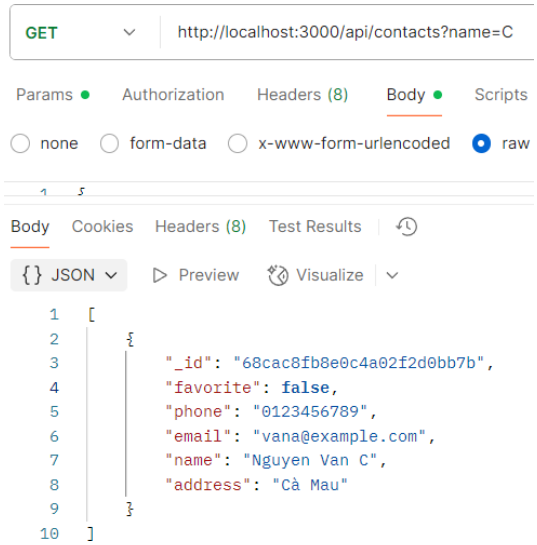


Hình 49: Test thử findAll

- contactService.find(condition*)* và *contactService.findByName(name)* lần lượt tìm kiếm các tài liệu thỏa điều kiện chỉ định trong đối tượng condition, theo tên name. Hai phương thức này có thể được định nghĩa như sau trong tập tin app/services/contact.service.js

```
async find(filter) {
    const cursor = await this.Contact.find(filter);
    return await cursor.toArray();
}

async findByName(name) {
    return await this.find({
        name: { $regex: new RegExp(name), $options: "i" },
    });
}
```

Hình 50: contactService.find(condition) và contactService.findByName(name)



Hình 51: Test tìm theo tên

➢ Cài đặt handler findOne trong tập tin app/controllers/contact.controllerjs như sau:

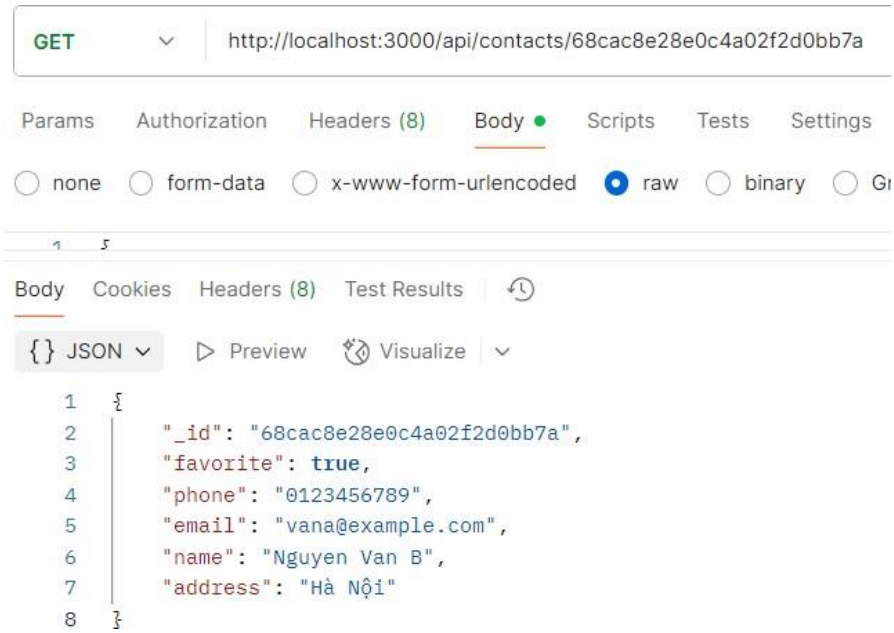
```
// Find a single contact with an id
exports.findOne = async (req, res, next) => {
    try {
        const contactService = new ContactService(MongoDB.client);
        const document = await contactService.findById(req.params.id);

        if (!document) {
            return next(new ApiError(404, "Contact not found"));
        }
        return res.send(document);
    } catch (error) {
        return next(
            new ApiError(
                500,
                `Error retrieving contact with id=${req.params.id}`
            )
        );
    }
};
```

Hình 52: Cài đặt handler findOne

➢ contactService.findByld*(id)* tìm kiếm tài liệu theo Id. Phương thức findByld*(id)* có thể được định nghĩa như sau trong tập tin *app/*services/contact.service.js

```
async findById(id) {
    return await this.Contact.findOne({
        _id: ObjectId.isValid(id) ? new ~~ObjectId~~(id) : null,
    });
}
```

Hình 53: Phương thức findByld(id)



Hình 54: Test tìm theo id

➢ Cài đặt handler update trong tập tin app/controllers/contact.controllerjs như sau:

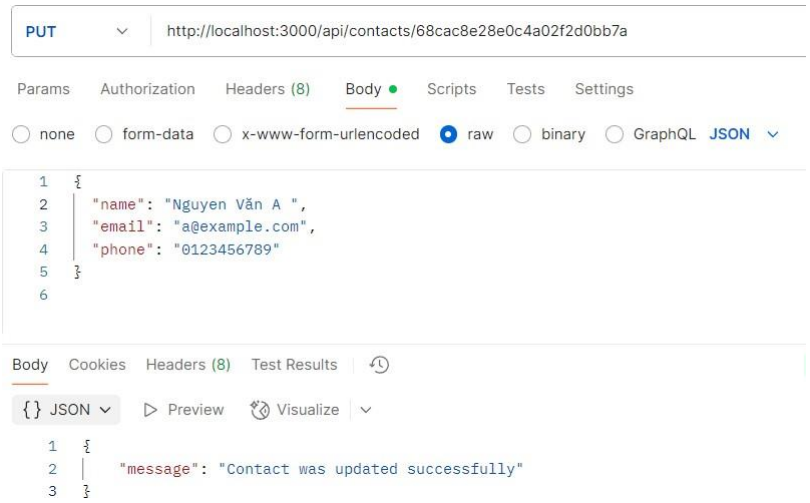
```
// Update a contact by the id in the request
exports.update = async (req, res, next) => {
    if (Object.keys(req.body).length === 0) {
        return next(new ApiError(400, "Data to update can not be empty"));
    }

    try {
        const contactService = new ContactService(MongoDB.client);
        const document = await contactService.update(req.params.id, req.body);

        if (!document) {
            return next(new ApiError(404, "Contact not found"));
        }

        return res.send({ message: "Contact was updated successfully" });
    } catch (error) {
        return next(
            new ApiError(
                500,
                `Error updating contact with id=${req.params.id}`
            )
        );
    }
};
```

Hình 55: Cài đặt handler update

➢ *contactService.update(id, document)* tìm kiếm tài liệu theo Id và cập nhật tài liệu này với dữ liệu trong đối tượng *document.* Phương thức update(id, *document)* có thể được định nghĩa như sau trong tập tin *app/services/contact.service.js*

```
  async update(id, payload) {
    const filter = { _id: ObjectId.isValid(id) ? new ~~ObjectId~~(id) : null };
    const update = this.extractContactData(payload);

    if (!update || Object.keys(update).length === 0) {
        return null; // không có gì để update
    }

    // Thực hiện update
    const result = await this.Contact.updateOne(filter, { $set: update });

    // Nếu không tìm thấy document nào để update
    if (!result || result.matchedCount === 0) {
        return null;
    }
    return await this.findById(id);
}
```

Hình 56: Phương thức update(id, document)



Hình 57: Test update

➢ Cài đặt handler delete trong tập tin app/controllers/contact.controllerjs như sau

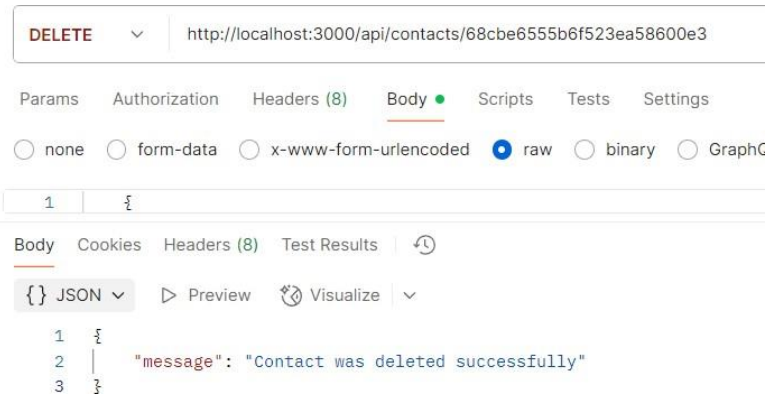
```
// Delete a contact with the specified id in the request

exports.delete = async (req, res, next) => {
  try {
    const contactService = new ContactService(MongoDB.client);
    const deletedDoc = await contactService.delete(req.params.id);

    console.log("DELETE handler result:", deletedDoc);

    if (!deletedDoc) {
      return next(new ApiError(404, "Contact not found"));
    }

    return res.send({ message: "Contact was deleted successfully" });
  } catch (error) {
    return next(
      new ApiError(500, `Could not delete contact with id=${req.params.id}`)
    );
  }
};
```

Hình 58: Cài đặt handler delete

➢ contactService.delete*(id)* tìm kiếm tài liệu theo Id và xóa tài liệu này. Phương thức delete*(id)* có thể được định nghĩa như sau trong tập tin app/*services*/contact.service.js.

```
async delete(id) {
    const filter = {
        _id: ObjectId.isValid(id) ? new ~~ObjectId~~(id) : null,
    };
    const result = await this.Contact.findOneAndDelete(filter);
    // Trả về document trực tiếp (không phải result.value)
    return result;
}
```

Hình 59: Phương thức delete(id)



Hình 60: Test Delete

➢ Cài đặt handler findAllFavorite trong tập tin app/controllers/contact.controllerjs như sau:

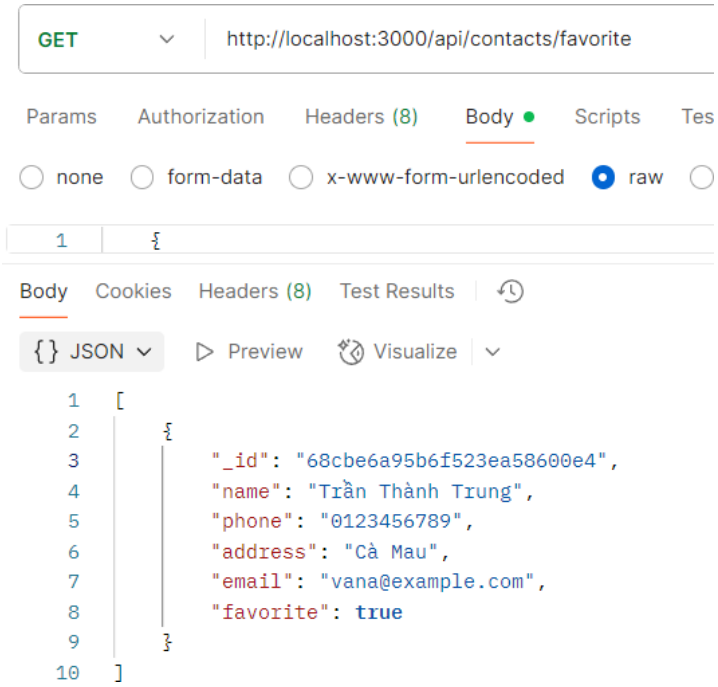
```
// Find all favorite contacts
exports.findAllFavorite = async (_req, res, next) => {
    try {
        const contactService = new ContactService(MongoDB.client);
        const documents = await contactService.findFavorite();
        return res.send(documents);
    } catch (error) {
        return next(
            new ApiError(
                500,
                "An error occurred while retrieving favorite contacts"
            )
        );
    }
};
```

Hình 61: Cài đặt handler findAllFavorite

➢ Phương thức findFavorite() trong lớp ContactService có thể được định nghĩa như sau có thể được định nghĩa như sau trong tập tin app/services/contact.service.js

```
async findFavorite() {
    return await this.find({ favorite: true });
}
```

Hình 62: Phương thức findFavorite()



Hình 63: Test findFavorite

➢ Cài đặt handler deleteAll

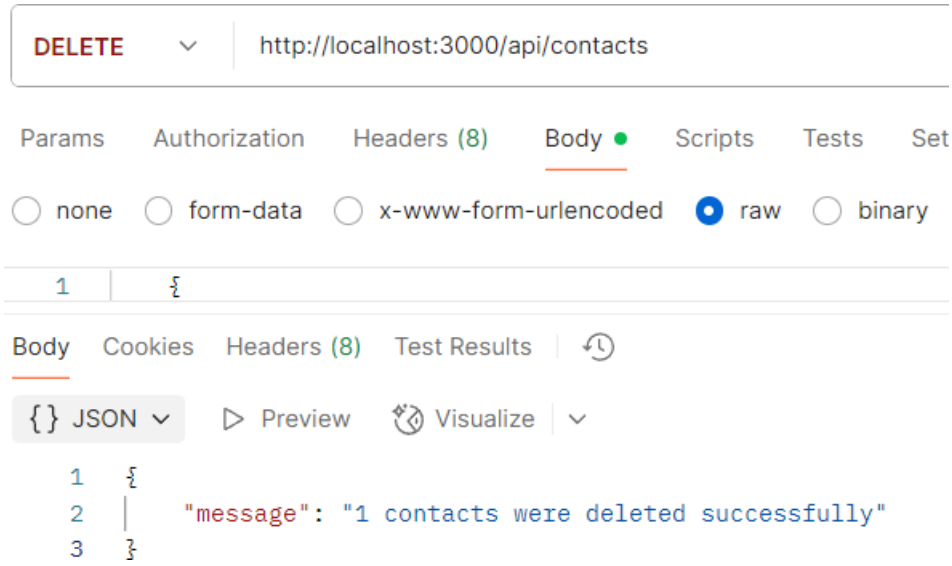
```
// Delete all contacts of a user from the database
exports.deleteAll = async (_req, res, next) => {
    try {
        const contactService = new ContactService(MongoDB.client);
        const deletedCount = await contactService.deleteAll();
        return res.send({
            message: `${deletedCount} contacts were deleted successfully`,
        });
    } catch (error) {
        return next(
            new ApiError(500, "An error occurred while removing all contacts")
        );
    }
};
```

Hình 64: Cài đặt handler deleteAll

➢ contactService.deleteMany*()* xóa tất các các đối tượng trong collection. Phương thức deleteAll*()* có thể được định nghĩa như sau

```
    async deleteAll() {
        const result = await this.Contact.deleteMany({});
        return result.deletedCount;
    }
}
```

Hình 65: Phương thức deleteAll()



Hình 66: Test deleteAll

➢ Gửi lên git

Hình 67: Lưu lên git

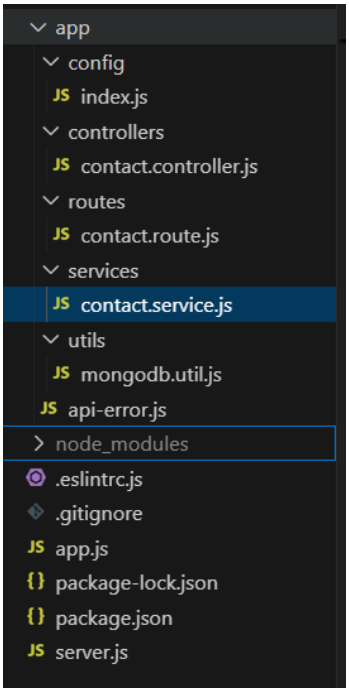
```
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (13/13), 2.83 KiB | 263.00 KiB/s, done.
Total 13 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/tungLuthanh/B1906607_LuThanhTung_Backend_1.git
   2d50852..e732084  main -> main
```

tungLuthanh Dinh nghia cac route cho tai nguyen Contact — cfc8331 · 2 minutes ago — **7 Commits**

app	Dinh nghia cac route cho tai nguyen Contact	2 minutes ago
.eslintrc.js	Cau hinh ESLint cho du an	19 minutes ago
.gitignore	Cai dat du an	1 hour ago
README.md	Initial commit	1 hour ago
app.js	Cai dat xu ly loi	1 hour ago
package-lock.json	Cai dat cac handler truy xuat CSDL	28 minutes ago
package.json	Cai dat cac handler truy xuat CSDL	28 minutes ago
server.js	Cai dat cac handler truy xuat CSDL	8 minutes ago

Hình 68: Đã lưu lên git

➢ Cấu trúc thư mục dự án đến thời điểm này sẽ như sau:



Hình 69: Cấu trúc thư mục dự án

ĐƯỜNG LINK :
https://github.com/tungLuthanh/B1906607_LuThanhTung_Backend_1.git